```
try {
    surpriseBoom();
} catch (DivisionByZeroError $e) {
    print "DIVIDE BY ZERO... OH SHI-" . PHP_EOL;
} catch (Error $e) {
    print "{$e->getMessage()} caught in generic Error handler" . PHP_EOL;
}

Output:
DIVIDE BY ZERO... OH SHI-
or...
Call to undefined function notDefined() caught in generic Error handler
```

The `surpriseBoom()` function is just a simple confection that effectively tosses a coin: 50/50 chance we'll end up either dividing by zero or calling an undefined function. Depending on the result of our logical expression, the function will accordingly throw either a `DivisionByZeroError` exception or a generic `Error`. The thing to notice here are the two `catch` blocks: both are defining a `$e` parameter but with different data types. When an exception is thrown in the `try` block, PHP will test the class of the thrown object against each `catch` block and execute the first matching type definition. This raises an important caveat: child classes count as their parents. If a type declaration says it will accept `Error`, then `DivisionByZeroError` will match it, because the child extends the parent (this is the *Liskov Substitution Principle* and is covered in precise detail in Chapter 10, "Programming with Objects"). What this means for us is that the most specialized class in the hierarchy must come first in the `catch` block order. If we'd put `catch (Error $e) { ...` first, then we'd never reach `catch (DivisionByZeroError $e)` because all `DivisionByZeroError` objects would satisfy the `Error` data type. Here's the proof:

```
try {
    throw new DivisionByZeroError("I divided by zero. :(");
} catch (Error $e) {
    print "{$e->getMessage()} caught in generic Error handler" . PHP_EOL;
} catch (DivisionByZeroError $e) {
    print "DIVIDE BY ZERO... OH SHI-" . PHP_EOL;
}
```